

Battery/Fuel-Cell Monitor BLE Wi-Fi Provisioning Protocol

Protocol: BLE Wi-Fi Provisioning Protocol

Version: 1.0

Target: ESP32 / ESP32-S3 using ESP-IDF NimBLE

Client: Battery Monitor Flutter application

Purpose: Configure and manage the Battery Monitor's Wi-Fi connection through BLE without requiring firmware recompilation.

1. Purpose

The Wi-Fi Provisioning Protocol allows a mobile application to configure Wi-Fi on the Battery/Fuel-Cell Monitor over BLE.

The protocol shall allow the application to:

- determine whether Wi-Fi has been configured;
- obtain the current Wi-Fi connection state;
- request a scan of nearby Wi-Fi networks;
- display those networks to the user;
- send an SSID and password to the Battery Monitor;
- instruct the Battery Monitor to save the credentials and connect;
- observe connection success or failure;
- replace previously stored credentials; and
- erase stored Wi-Fi credentials.

Wi-Fi provisioning shall remain independent of the measurement acquisition system.

Failure of BLE provisioning or Wi-Fi connectivity shall **never stop, delay, or control the 1 kHz acquisition pipeline**.

2. Existing BLE Services

The existing Battery Monitor GATT service remains unchanged.

Existing service:

None

Battery Monitor Service

5ecf0000-41c2-4cc4-9c96-640f406021d0

Existing characteristics:

None

Voltage Data

5ecf0001-41c2-4cc4-9c96-640f406021d0

System Status

5ecf0002-41c2-4cc4-9c96-640f406021d0

Configuration

5ecf0003-41c2-4cc4-9c96-640f406021d0

Command

5ecf0004-41c2-4cc4-9c96-640f406021d0

The new provisioning functionality shall use a **separate BLE service**.

This prevents Wi-Fi provisioning changes from affecting the already-working measurement BLE protocol. Pasted text

3. Wi-Fi Provisioning Service

Service UUID

None

5ecf1000-41c2-4cc4-9c96-640f406021d0

Characteristics

Characteristic	UUID	Properties	Purpose
----------------	------	------------	---------

Provisioning Control	5ecf1001-41c2-4cc4-9c96-640f406021d0	WRITE, NOTIFY	Commands, acknowledgments, errors
Provisioning Data	5ecf1002-41c2-4cc4-9c96-640f406021d0	WRITE, NOTIFY	Fragmented credentials and scan results
Wi-Fi Status	5ecf1003-41c2-4cc4-9c96-640f406021d0	READ, NOTIFY	Current Wi-Fi state

The app should subscribe to notifications from:

```
None
Provisioning Control
Provisioning Data
Wi-Fi Status
```

immediately after discovering the provisioning service.

4. Protocol Version

All protocol messages begin with:

```
None
Protocol Version = 0x01
```

A future incompatible protocol shall increment this value.

If either side receives an unsupported protocol version, it shall reject the request.

5. Wi-Fi States

These values shall match the states already implemented in `WiFiManager`.

Value	State	Meaning
0x00	UNPROVISIONED	No usable credentials are configured
0x01	CONNECTING	ESP is attempting Wi-Fi connection
0x02	CONNECTED	ESP obtained an IP address
0x03	CONNECTION_FAILED	Connection attempt failed

The BLE layer shall query the **WiFiManager**; it shall not maintain a second independent Wi-Fi state machine.

6. Wi-Fi Status Characteristic

UUID:

None

5ecf1003-41c2-4cc4-9c96-640f406021d0

Properties:

None

READ

NOTIFY

The value is a fixed **8-byte structure**:

Byte	Field
0	Protocol version
1	Wi-Fi state
2	Flags
3	Last error code

- 4 IPv4 byte 1
- 5 IPv4 byte 2
- 6 IPv4 byte 3
- 7 IPv4 byte 4

Flags

Byte 2:

None

Bit 0 = credentials stored
Bit 1 = Wi-Fi scan active
Bit 2 = provisioning transaction active
Bit 3 = BLE link encrypted
Bits 4-7 = reserved

Example:

None

01 02 09 00 C0 A8 01 2A

means:

None

Version: 1
State: CONNECTED
Credentials: Yes
Encrypted BLE: Yes
Error: None
IPv4: 192.168.1.42

When no valid IPv4 address exists:

None

0.0.0.0

shall be reported.

The ESP shall send a status notification whenever the Wi-Fi state changes.

For example:

None
UNPROVISIONED
↓
CONNECTING
↓
CONNECTED

or:

None
CONNECTING
↓
CONNECTION_FAILED

7. Provisioning Control Characteristic

UUID:

None
5ecf1001-41c2-4cc4-9c96-640f406021d0

Properties:

None
WRITE
NOTIFY

Commands use:

None

Byte 0 Protocol version
Byte 1 Opcode
Byte 2 Transaction ID
Byte 3 Payload length
Byte 4+ Payload

All control messages shall fit within a normal BLE notification/write without requiring fragmentation.

The **Transaction ID** is selected by the application and allows responses to be associated with a specific operation.

Transaction IDs may use:

None

0x01–0xFF

0x00 is reserved for unsolicited ESP notifications.

8. Command Opcodes

Opcode	Command
0x01	GET_STATUS
0x02	START_SCAN
0x03	BEGIN_CREDENTIALS
0x04	COMMIT_CREDENTIALS
0x05	CLEAR_CREDENTIALS

0x06 CANCEL

GET_STATUS — 0x01

Requests an immediate Wi-Fi Status notification.

Payload:

None
none

The app can normally read the Status characteristic directly, so this command is optional but useful for synchronization.

START_SCAN — 0x02

Requests that the ESP32 scan nearby Wi-Fi networks.

Payload:

None
none

Expected sequence:

None

App	ESP
START_SCAN -->	
<---- ACK ---->	
<-- results ---	
<-- results ---	
<-- results ---	

```
|  
|<- SCAN_DONE --|
```

The Wi-Fi scan shall execute in the communications subsystem.

It shall not suspend the acquisition task.

BEGIN_CREDENTIALS — 0x03

Starts a credential provisioning transaction.

The ESP shall:

1. discard any previous incomplete staged credential data;
2. create a temporary RAM buffer;
3. remember the transaction ID;
4. wait for a credential object over the Provisioning Data characteristic.

This command **does not immediately modify NVS**.

COMMIT_CREDENTIALS — 0x04

Tells the ESP that all credential data has been transmitted.

The ESP shall:

1. validate the staged credential object;
2. save it using the existing Wi-Fi NVS API;
3. clear sensitive staging buffers when they are no longer required;
4. set Wi-Fi state to **CONNECTING**;
5. initiate connection;
6. notify the app of state changes.

Credentials are stored using the existing:

None

namespace: wifi_cfg

ssid

password

keys.

CLEAR_CREDENTIALS — 0x05

Deletes stored Wi-Fi credentials using the existing `WiFiManager` credential-clearing API.

After completion:

None

WiFiState = UNPROVISIONED

Wi-Fi shall disconnect if currently connected.

Acquisition shall continue normally.

CANCEL — 0x06

Cancels the active provisioning transaction.

The ESP shall:

- discard staged credentials;
 - clear temporary credential memory;
 - stop the provisioning transaction;
 - leave previously stored NVS credentials unchanged.
-

9. Control Responses

ESP responses use the same Provisioning Control characteristic.

ACK

Opcode:

None
0x80

Format:

None
Byte 0 Version
Byte 1 0x80
Byte 2 Transaction ID
Byte 3 2
Byte 4 Original command opcode
Byte 5 Result code

Example:

None
01 80 17 02 02 00

means:

None
Transaction: 0x17
Command: START_SCAN
Result: OK

ERROR

Opcode:

None

0x81

Format:

None

Byte 0 Version

Byte 1 0x81

Byte 2 Transaction ID

Byte 3 2

Byte 4 Original command opcode

Byte 5 Error code

SCAN COMPLETE

Opcode:

None

0x82

Payload:

None

Byte 4 = number of networks reported

10. Error Codes

Code	Meaning
0x00	OK
0x01	Unsupported protocol version
0x02	Invalid command

- 0x03 Invalid payload
- 0x04 Operation busy
- 0x05 BLE security requirement not met
- 0x06 Wi-Fi scan failed
- 0x07 Invalid SSID
- 0x08 Invalid password
- 0x09 NVS save failed
- 0x0A Wi-Fi connection failed
- 0x0B Connection timeout
- 0x0C No staged credentials
- 0x0D Transaction ID mismatch
- 0x0E Fragment sequence/length error
- 0x0F Internal error

Do not expose the password as part of an error message or diagnostic log.

11. Provisioning Data Characteristic

UUID:

None

5ecf1002-41c2-4cc4-9c96-640f406021d0

Properties:

None

WRITE

NOTIFY

Variable-sized objects are fragmented so provisioning **does not depend on negotiating a large ATT MTU**.

This is useful because your existing 80-byte voltage notification currently relies on an ATT MTU of roughly 83 bytes, which is already identified as something that should eventually be made more robust. Pasted text

Each Data packet uses:

Byte	Field
0	Protocol version
1	Message type
2	Transaction ID
3	Object ID
4	Offset
5	Total object length
6	Chunk length
7+	Chunk data

At the default ATT MTU of 23:

None

20-byte characteristic payload
- 7-byte protocol header
= 13 bytes of data per fragment

This is intentionally acceptable.

12. Provisioning Data Message Types

Value	Type
0x01	WIFI_CREDENTI ALS
0x02	SCAN_RESULT
0x03	NETWORK_INFO

13. Credential Object

Message type:

```
None
0x01 WIFI_CREDENTIALS
```

The complete reconstructed object is:

```
None
Byte 0      SSID length
Byte 1      Password length
Bytes ...   SSID
Bytes ...   Password
```

Limits:

```
None
SSID:       0-32 bytes
Password:   0-63 bytes
```

For an open network:

```
None
password_length = 0
```

Example:

None

SSID:

LabWiFi

Password:

example123

becomes conceptually:

None

07 0A

4C 61 62 57 69 46 69

65 78 61 6D 70 6C 65 31 32 33

The credential object may arrive across multiple BLE writes.

The ESP shall not process it until all bytes have arrived and `COMMIT_CREDENTIALS` is received.

14. Fragment Reassembly

The receiving side shall verify:

None

Transaction ID matches

Total length remains constant

Offset is valid

Chunk length is valid

$\text{offset} + \text{chunk_length} \leq \text{total_length}$

Duplicate fragments may be ignored if the same data has already been accepted.

Malformed fragments shall cause:

None

`ERROR = FRAGMENT_ERROR`

Incomplete provisioning data shall automatically expire after approximately:

None

`60 seconds`

Sensitive temporary buffers shall then be cleared.

15. Wi-Fi Scan Results

Each discovered network is sent as one `SCAN_RESULT` object, fragmented if necessary.

Object format:

None

Byte 0	Result index
Byte 1	RSSI, signed int8
Byte 2	Authentication type
Byte 3	SSID length
Byte 4+	SSID

Example:

None

```
Index:    2
RSSI:     -48 dBm
Security: WPA2
SSID:     HomeWiFi
```

The application should sort networks by signal strength if desired.

Duplicate SSIDs may be consolidated by the app.

16. Authentication Type Values

Value	Type
0x00	Open
0x01	WEP
0x02	WPA Personal
0x03	WPA2 Personal
0x04	WPA/WPA2 Personal
0x05	WPA3 Personal
0x06	WPA2/WPA3 Personal
0x07	Enterprise
0xFF	Unknown

For protocol version 1, the intended supported networks should be:

None
Open
WPA2 Personal
WPA3 Personal where supported
WPA2/WPA3 transition networks

Enterprise/campus authentication may be shown by the application as:

None
Unsupported in this version

rather than silently attempting an incompatible password flow.

17. Network Information Object

After a successful connection, the ESP may send:

```
None
0x03 NETWORK_INFO
```

Object:

```
None
Byte 0      SSID length
Byte 1...   SSID
Next 4      IPv4 address
Final byte  RSSI
```

This gives Will's app enough information to display:

```
None
Connected to: HomeWiFi
IP address:   192.168.1.42
Signal:      -47 dBm
```

The password shall never be returned.

18. Required App Workflow

The intended Flutter app workflow is:

```
None
App opens
  |
  v
Scan for BatteryMonitor
  |
```

```

    v
Connect BLE
    |
    v
Discover Wi-Fi Provisioning Service
    |
    v
Read Wi-Fi Status
    |
    +----- CONNECTED -----+
    |                               |
    |                               | Show normal device UI
    |                               |
    +----- UNPROVISIONED
    |
    v
Prompt:
"Set up Wi-Fi?"
    |
    v
START_SCAN
    |
    v
Display nearby networks
    |
    v
User selects SSID
    |
    v
User enters password
    |
    v
BEGIN_CREDENTIALS
    |
    v
Send credential fragments
```



19. Changing Wi-Fi Later

The application shall provide:

```
None
Settings
└─ Wi-Fi
    │─ Current network
    │─ Change network
    └─ Forget network
```

Selecting **Change network** does not require clearing credentials first.

The new credentials can simply replace the old ones after a valid provisioning transaction.

Selecting **Forget network** sends:

```
None
CLEAR_CREDENTIALS
```

20. Connection Failure Behavior

If connection fails:

```
None  
WiFiState = CONNECTION_FAILED
```

The app should display an actionable message such as:

```
None  
Unable to connect to HomeWiFi.  
  
Check the password and make sure the network  
is available, then try again.
```

The ESP shall **not reboot**.

It shall **not stop acquisition**.

It shall **not disable BLE**.

The app must remain capable of sending replacement credentials.

21. BLE Security

Wi-Fi passwords shall not be accepted over a completely unsecured BLE provisioning connection in the final system.

Sensitive commands are:

```
None  
BEGIN_CREDENTIALS  
WIFI_CREDENTIALS data  
COMMIT_CREDENTIALS
```

CLEAR_CREDENTIALS

These should require an encrypted BLE connection.

For the final product, the preferred configuration is:

None

BLE LE Secure Connections

+

authenticated pairing

A strong future implementation would display a pairing/passkey code on the OLED so the user can confirm that the phone is configuring the physical Battery Monitor in front of them.

During development, a temporary less-strict mode may be useful for testing, but it should be explicitly compile-time controlled and disabled for final demonstration firmware.

22. Logging Requirements

Firmware may log:

None

Provisioning transaction started

Wi-Fi scan started

Wi-Fi scan complete: N networks

Credentials received

Credentials saved successfully

Connecting to SSID: HomeWiFi

Wi-Fi connected

Wi-Fi connection failed

Credentials cleared

Firmware shall never log:

None

Wi-Fi password
raw credential object
credential fragments containing password bytes

23. Concurrency Requirements

Provisioning shall run entirely outside the acquisition ISR/task.

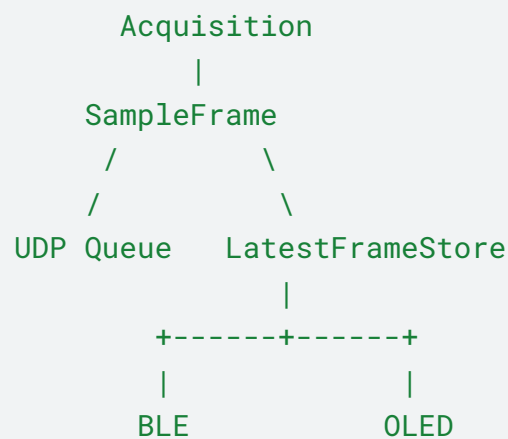
The provisioning implementation shall not:

None

read the ADC
modify SampleFrame
consume the acquisition queue
block AcquisitionTask
change timer configuration
change the 1 kHz acquisition rate
modify UDP packet serialization

The desired architecture is:

None



Wi-Fi Provisioning



Provisioning interacts with `WiFiManager`, not with acquisition.

24. ESP32 Responsibility

Your firmware is responsible for:

- None
- Provisioning GATT service
- Provisioning state/transaction handling
- Wi-Fi scanning
- Fragment reassembly
- Credential validation
- Calling `saveCredentials()`
- Calling `clearCredentials()`
- Starting Wi-Fi connection
- Publishing Wi-Fi status notifications
- NVS persistence
- Error handling
- Credential-memory cleanup
- BLE security enforcement

25. Will's Flutter App Responsibility

Will's app is responsible for:

None

- BLE discovery
- Connecting to BatteryMonitor
- Discovering provisioning characteristics
- Subscribing to notifications
- Reading Wi-Fi Status
- Prompting the user when unprovisioned
- Requesting network scans
- Reassembling scan-result fragments
- Displaying SSIDs
- Collecting the password
- Fragmenting the credential object
- Sending provisioning commands
- Displaying CONNECTING / CONNECTED / FAILURE
- Allowing network changes
- Allowing credentials to be forgotten

This gives you a very clean subsystem boundary.

26. Version 1 Out of Scope

The following should **not** be part of provisioning v1:

None

- Captive portal
- SoftAP provisioning
- WPA Enterprise credential provisioning
- Campus certificate management
- mDNS discovery
- UDP destination discovery
- Cloud account setup
- Database authentication
- Firmware OTA
- ADC configuration
- Calibration

Ethernet configuration
CAN configuration

These can be independent later stages.

27. Acceptance Tests

Both your firmware and Will's app should eventually pass these:

Test	Expected Result
Fresh device	App sees UNPROVISIONED and offers setup
Wi-Fi scan	Nearby networks appear
Correct password	State goes CONNECTING → CONNECTED
Power cycle	Device automatically reconnects using NVS
Wrong password	App receives failure; acquisition continues
Router unavailable	Wi-Fi fails/retries; acquisition continues
Router restored	Existing reconnect behavior resumes connectivity
Change Wi-Fi	New credentials replace old credentials
Forget Wi-Fi	Device becomes UNPROVISIONED
BLE disconnect during provisioning	Incomplete staged credentials expire safely
Malformed fragments	Transaction rejected without crash
Wi-Fi failure	~1,000 SampleFrames/s remains intact
Provisioning operation	No acquisition queue overflow caused by provisioning

The most important design decision

